

# Combining Obfuscation Specialists: What Routing and Merging Buy for Models That Read Transformed Code

Anonymous Author(s)

## ABSTRACT

Software engineers routinely read code that has been mechanically transformed: minified and bundled JavaScript, packed release artifacts, samples under malware triage. Fine-tuning a code model on one such transformation reliably helps on that transformation and almost nowhere else, so the natural engineering response is modular — train one small adapter per transformation and combine them. We evaluate that response directly. On a ladder of six semantics-preserving transformations in Python and JavaScript, with a seventh obfuscator quarantined behind four independent enforcement layers, we build and measure four ways of combining six LoRA specialists: a learned type router, three weight-space merges, an activation-space mixture over a frozen expert bank, and oracle prompting. We report three results that hold today. A router that identifies the transformation with 100.0% accuracy and near-zero predictive entropy is statistically indistinguishable from a single adapter trained on clean code on five of six conditions — perfect dispatch is worth almost nothing, because there is almost nothing to dispatch to. A greedy search over expert checkpoints, 27 candidate merges across three rounds, moves merged accuracy by at most 0.7 points. And the one arm previously reported as catastrophic was a rescaling defect, not an algorithmic result: repaired, it sits at parity with the control. **[CLAIM SLOT: headline — see CLAIM\_LADDER.md; the mixture arm’s decisive control is currently inert (§7) and the effect it would adjudicate is +2.9 points at  $p = 0.06$ .]** We release the transformation ladder, the quarantine architecture, and a CPU-only task-vector diagnostic that predicts merge interference without a GPU.

## KEYWORDS

code obfuscation, program comprehension, model merging, parameter-efficient fine-tuning, mixture of experts, negative results

## 1 INTRODUCTION

Reading transformed code is ordinary software engineering work. Front-end engineers debug minified bundles; release engineers read stripped binaries and packed artifacts; security analysts triage samples that have been deliberately obfuscated. In each case the code still means what it meant, and a competent engineer can still say what it computes — more slowly, and less reliably [7]. Language models show the same profile, and degrade on roughly the same inputs humans do [5].

The obvious remedy is to fine-tune. It works, narrowly. In the study this paper builds on, an adapter trained on one transformation improves on that transformation by 1 to 8 accuracy points and transfers essentially nowhere else: the mean off-diagonal transfer ratio across a 42-cell matrix is 0.073, several cross-family cells are negative, and an adapter trained on all five transformations at once is *worse* than one trained on clean code on a held-out obfuscator.

Fine-tuning on obfuscated code teaches inversion of the specific transformations seen, not invariance to the class.

That leaves a concrete engineering question, and it is the question this paper asks:

**If each transformation needs its own specialist, can the specialists be combined into one deployable system that keeps what each of them knows?**

This is not a hypothetical. It is the design decision anyone shipping such a model actually faces, and the literature offers two families of answer. *Routing* keeps every adapter and picks one per input [3, 9]. *Merging* averages the adapters into a single set of weights, so nothing has to be picked at inference time [4, 12, 14, 15]. A third option sits between them: keep the experts resident and blend them per token in activation space [6, 13]. We build all three, plus the cheap baseline of simply telling an untuned model which transformation was applied.

*This draft’s status.* This is an honest interim report, and the structure reflects that. Sections 4–6 carry results we consider settled. Section 7 carries an effect of the right sign on every condition that does not clear correction, alongside a control we discovered to be inert while preparing this draft. Section 9 specifies the experiments that would resolve it, each with the decision gate it feeds. Unmeasured quantities appear as – throughout and are never filled with plausible values; unearned claims appear as **[CLAIM SLOT: ]**.

### Contributions.

- (1) A controlled comparison of four adapter-combination strategies on a program-comprehension task, on two languages, against the reference that matters — an adapter trained on *clean* code, not an untuned base model (§3).
- (2) The finding that a *saturated* router is a ceiling rather than a success: at 100.0% routing accuracy, learned and oracle routing are the same system, and it buys the specialists’ own gains and nothing more (§4).
- (3) A negative result for merge-objective checkpoint selection: 27 candidate merges over three greedy rounds span 0.7 accuracy points (§5).
- (4) A CPU-only task-vector diagnostic, exact for vanilla LoRA, that measures inter-expert interference from the  $r \times r$  factors without materialising a dense  $\Delta W$  (§6), and the finding that the interference mechanism reported in prior work reproduces while its accuracy consequence does not.
- (5) A four-layer quarantine architecture for held-out transformations, and a candid account of how its epistemic guarantee degraded in practice (§8).
- (6) **[CLAIM SLOT: the mixture result, pending §9 item 5]**

## 2 BACKGROUND AND RELATED WORK

*Obfuscation and comprehension.* Obfuscation applies semantics-preserving transformations to make code harder to understand. Its effect on human comprehension is measurable and transformation-dependent [7], and large language models misread broadly the same inputs [5]. Most machine-learning work on obfuscated code targets *recovery* — deobfuscation, decompilation, identifier restoration [8, 10, 11]. We do not. Every system here is trained and evaluated on code that stays obfuscated, and is scored on whether it can say what the code computes. That separation is deliberate: recovery quality and comprehension are different questions, and a model can copy its input forward and score well on several recovery metrics without having understood anything [8].

*Parameter-efficient specialists.* LoRA [2] makes per-transformation specialists cheap: each is a low-rank patch of about 0.5% of model size beside a frozen base. Six specialists are one model and six small files. The question is what to do with the six.

*Merging.* Task arithmetic treats a fine-tuned model as base plus a *task vector* and adds vectors to compose skills [4]; model soups average weights directly [12]. Both degrade when task vectors conflict. TIES [14] prunes low-magnitude coordinates and resolves sign disagreements by electing the side with greater total magnitude; DARE [15] randomly drops and rescales updates before merging. Recent work attributes merge failure to *over-training* the experts: fine-tuning to an individual optimum is dominated late in training by memorisation of hard examples, producing negative parameter interference, with aggressive early stopping as the recommended fix [1]. §6 tests that mechanism directly and finds it present but inconsequential here.

*Routing and mixtures.* AdapterFusion learns to compose trained adapters [9]; LoRAHub composes them by black-box optimisation for an unseen task [3]; mixture-of-LoRA architectures gate experts per token and per layer [6, 13]. These methods are usually evaluated where the routing decision is hard. Our setting inverts that: §4 shows the decision is trivially easy and therefore uninformative, which is what pushes the interesting question to inputs where *no* single expert is correct (§7).

*Positioning.* What we add is not a new merging algorithm. It is a controlled measurement, on a software-engineering task with an execution-verified grader, of what each combination strategy is worth against the right reference — and an account of which of our own arms we could not yet trust.

## 3 STUDY DESIGN

### 3.1 Task and grader

The task is *output prediction*: given a self-contained function and one concrete input, predict the exact return value, no explanation. Grading runs the original program to obtain ground truth and requires strict normalised exact match — no containment or substring matching, which an earlier audit found produces roughly 3% false positives (927 matching inside 9273). There is no grader subjectivity and no LLM judge. We additionally report the *format-failure rate*, the fraction of answers that cannot be parsed into a candidate value at all, because a system that stops answering and

**Table 1: The condition ladder. H1 is quarantined and never trained on by anything.**

| code  | family     | transformation                                           |
|-------|------------|----------------------------------------------------------|
| L0    | —          | clean original (comments/docstrings stripped everywhere) |
| L1b   | identifier | adversarial renaming: actively misleading names          |
| L1r   | identifier | random hex renaming (v_a3f2)                             |
| L2    | identifier | sequential minification (a, b, ...)                      |
| S1    | structural | control-flow flattening (dispatch loop)                  |
| S2    | structural | opaque predicates + dead code                            |
| S3/S4 | structural | the two halves of S2 alone                               |
| H1    | held out   | string encoding + guarded MBA rewriting                  |

a system that answers wrongly are different failures (§5 turns on exactly that distinction).

### 3.2 The transformation ladder

Conditions are *single* transformations from the clean parent, never stacked, with identical semantics in Python and JavaScript. Every variant is verified to preserve semantics by executing it against its parent’s inputs plus roughly 20 fuzzed ones.

*Composite* conditions (C\_L1r\_S1 = rename then flatten, and five others) stack two transformations. They sit outside the trainable ladder by construction and exist for one reason: they are the only inputs where no single expert is correct, which is what §7 needs.

### 3.3 Quarantine

H1 is the discriminator: a system that handles it learned the *class* of meaning-preserving rewrites, one that does not memorised the transformations it saw. If it leaks into training, no downstream analysis can recover the claim, so enforcement does not rely on discipline. Four independent layers, because any one alone is by-passable: (i) a single training-read entry point that raises on any path outside the training tree and rejects H1-labelled rows wherever found; (ii) a lint test that greps the source for raw file reads bypassing that entry point; (iii) a generator that refuses to run without an explicit flag and writes its outputs read-only; (iv) a manifest checker that verifies SHAs *and* content-scans every training file for H1 markers, wired as a pre-commit hook — this last catches leakage even when labels are correct. §8 reports where this architecture nonetheless fell short of its own guarantee.

### 3.4 Two grids, never pooled

Results come from two evaluation program sets that are *disjoint* and are never averaged together: a **corpus** grid (557 Python / 168 JavaScript held-out programs) and a **test-set** grid (40 Python / 30 JavaScript). All combination arms live on the test-set grid. A number from one grid is never differenced against a number from the other. Headline numbers use the subset of programs surviving *every* condition, since the structural transformations decline some programs by design.

### 3.5 The reference, and why it is not the base model

Almost every delta we report is against an adapter trained *only on clean code*, not against the untuned model. This matters more than any individual number. The untuned base has a 17.3% format-failure rate in Python; adapters run at 1–3%. A large part of any base-relative gain is the model learning the answer *format*, not the task. On the test-set grid the base model sits 14 to 30 points below the clean-code control across conditions (Table 2, last row). **A system that beats the base model has shown nothing.**

### 3.6 Statistics

Confidence intervals come from a cluster bootstrap resampling whole *programs* with replacement (2000 draws, seed 17): several evaluation items share a program and are correlated, so bootstrapping items would understate the intervals. We apply Benjamini–Hochberg FDR within each family of comparisons, and call an effect only when  $q < 0.05$  and the bootstrap interval excludes zero — both, not either. Every comparison is restricted to programs present in both arms, so no delta is confounded by a different program population. Every kept run records a manifest with its command, resolved config, script hashes, commit, GPU and timestamp.

Base model: Qwen2.5-Coder-1.5B-Instruct throughout, seed 17, greedy decoding. §10 treats the single-model and single-seed limitations.

## 4 A PERFECT ROUTER BUYS ALMOST NOTHING

### 4.1 What was built

The router is a classifier over frozen features: the base model’s hidden state at a mid-depth layer, mean-pooled over non-special prompt tokens, feeding a one-hidden-layer MLP (512 units, GELU, dropout 0.1) with eight classes. **H1 is never a class**, by design. At inference each item is routed independently and the selected adapter is served through a multi-LoRA path.

### 4.2 The router is saturated

It classifies the transformation with 100.0% accuracy on every evaluated item in both languages — 1,025 Python items and 804 JavaScript items, a completely diagonal confusion matrix — with held-out validation accuracy 0.9969 (Python) and 1.0000 (JavaScript). It is also extremely confident: mean predictive entropy runs from  $3.9 \times 10^{-7}$  to  $6.7 \times 10^{-3}$  nats against a maximum of  $\ln 8 = 2.079$ , i.e. it places at least 99.9% of its mass on the chosen class in every condition.

This is unsurprising on inspection. The conditions leave unmistakable surface fingerprints: hex identifiers for L1r, single letters for L2, a state-variable dispatch loop for S1. A mid-depth hidden state separates them trivially.

### 4.3 Why saturation makes the arm uninformative

Two consequences follow, and the second is the finding.

First, **learned routing and oracle routing are the same system here**. This arm cannot tell us whether a router is *learnable* in a harder setting; it can only tell us what perfect dispatch is worth.

Second, perfect dispatch is worth very little. Table 2 gives the answer: against the clean-code control the routed system is +0.6, +0.6, +0.6, +0.0 and −1.7 points on L0, L1r, L2, S1 and S2 — five of six conditions indistinguishable from a single adapter that never saw an obfuscated program. Only L1b moves (+9.1, [+1.2, +16.7]), and it does not survive correction ( $q = 0.056$ ). The routed system’s numbers are, cell for cell, the numbers of whichever specialist it dispatched to. **Routing recovers exactly what the specialists already knew and adds nothing on top**; since the specialists individually transfer nowhere, a perfect router inherits that limitation.

### 4.4 What the router does out of distribution is not yet known

The configuration declares a routing-entropy analysis on H1 — the idea being that a held-out transformation forces an out-of-distribution decision, and *how* the router fails (confidently wrong versus uncertain) is itself informative for anyone deploying one. The routing report records zero held-out items. H1 was never routed, so this analysis does not exist: see Table 4, item 6.

## 5 MERGING LOSES WHAT ROUTING PRESERVES

### 5.1 What was built

All merges combine the *same* six ingredients — the L0, L1b, L1r, L2, S1 and S2 specialists, rank 32, seed 17 — into one adapter with uniform weights (1/6) and density 0.5, and differ only in the combination algorithm. Three implementation details determine whether the numbers mean anything:

- (1) **Merging happens in LoRA space, not in base weights.** A widely used merging toolkit was tried and rejected: its LoRA path merges each adapter into the base model and re-extracts a LoRA by SVD, changing the rank and injecting reconstruction error *before* the merge algorithm runs. We operate on the adapters directly.
- (2) **Each ingredient’s scaling is accounted for.** The specialists are  $r=32$ ,  $\alpha=64$  (scaling 2.0); the merged adapter is written with  $\alpha = r$  and the factor baked into the weights. This is the classic place where a merge silently halves every ingredient.
- (3) **H1 cannot be an ingredient.** The merge driver rejects any non-trainable condition by label and any adapter whose path touches the quarantine tree. A merge containing an H1-trained adapter would turn every downstream H1 number into a train-on-test result with nothing in the trial table to reveal it.

### 5.2 Results

Table 2. `merge_dare_ties`, the best of the merges, is statistically indistinguishable from the clean-code control on all six trainable conditions and −5.2 [−11.7, +0.0] on the held-out obfuscator. `merge_ties`, which omits DARE’s stochastic dropout, is significantly *below* the control on L1r, L2, S2 and H1. Collapsing six adapters into one destroys most of what they individually knew: compare the routed row, which at least recovers its specialists.

**Table 2: Combination strategies against the clean-code control, test-set grid, Python, in accuracy points. \* = BH-FDR  $q < 0.05$  and cluster-bootstrap CI excludes zero. The H1 column is the held-out obfuscator. router was never evaluated on H1 (§4); dl\_rescaled is the repaired arm and postdates the H1 read budget (§8). Absolute control accuracies: .494/.369/.460/.500/.434/.489/.400.**

| system            | L0     | L1b    | L1r    | L2     | S1     | S2     | H1     | what it is                                        |
|-------------------|--------|--------|--------|--------|--------|--------|--------|---------------------------------------------------|
| router            | +0.6   | +9.1   | +0.6   | +0.6   | +0.0   | -1.7   | –      | six specialists + a 100%-accurate classifier      |
| merge_dare_ties   | +0.0   | +2.8   | -4.0   | +0.0   | +0.7   | -4.5   | -5.2   | DARE drop-and-rescale, then TIES sign election    |
| dl_rescaled       | -2.3   | +2.8   | -1.1   | +1.1   | +0.7   | -2.8   | –      | DARE + linear, <i>after</i> the scaling repair    |
| merge_ties        | -8.0   | -5.1   | -11.9* | -11.4* | -4.8   | -16.5* | -11.3* | TIES alone, no DARE dropout                       |
| merge_dare_linear | -45.5* | -33.0* | -43.2* | -47.7* | -38.6* | -42.6* | -33.9* | <b>a defect, not a result</b> (§5)                |
| oracle_prompt     | -9.7   | -10.2* | -19.9* | -21.0* | -16.6* | -21.0* | -18.3* | untuned, <i>told</i> which transformation applies |
| base              | -21.0* | -14.2* | -23.3* | -28.4* | -16.6* | -30.1* | -28.7* | untouched pretrained model                        |

### 5.3 A catastrophic arm that was a defect

merge\_dare\_linear scored around 5% accuracy with a 53.5% format-failure rate in Python — on half its inputs the model stopped emitting a parseable answer at all, while the other merges sat at 1–3%. A -33 to -48 point collapse alongside a format-failure rate an order of magnitude off every other arm is a bug signature, not an algorithmic result, and we flagged it as one rather than reporting it as evidence about DARE.

It was a bug. Repaired, the same algorithm (dl\_rescaled) gains +41.6 points [+33.9, +49.9] pooled over the six conditions and lands at parity with the clean-code control on all of them, with format-failure back to 0.6–2.8%. We report this at length because the failure mode generalises: a merge that produces an incoherent patch degrades the *format* channel first, and a pipeline reporting only accuracy would have published a -40-point “finding” about DARE-linear merging.

### 5.4 Merge-objective checkpoint selection does not help

Our checkpoint selection picks each expert’s best epoch by its own held-in validation accuracy — an *individually* optimal choice that has no reason to be jointly optimal. If interference is the problem (§6), selecting checkpoints for the merge should beat it. We ran a greedy search: three rounds, each sweeping one expert’s checkpoint over nine epochs with the others held at the incumbent, 27 candidate merges evaluated on the full condition set.

The search is flat. Candidate means span 0.4434 to 0.4506 — 0.7 accuracy points end to end — and each round’s winner is *below* the status-quo merge by 0.7 to 1.0 points (all  $p > 0.4$ ). Choosing checkpoints for mergeability rather than for individual accuracy buys nothing here. We regard this as a genuine negative result about the objective, bounded by §10: greedy coordinate search over epochs is a coarse instrument, and it cannot rule out that some jointly-selected assignment outside its path does better.

## 6 TASK-VECTOR GEOMETRY: THE MECHANISM WITHOUT THE CONSEQUENCE

### 6.1 A diagnostic that needs no GPU

For vanilla LoRA the task vector is exactly  $\Delta W = (\alpha/r) BA$ , which makes inter-expert geometry computable from the  $r \times r$  factors without ever materialising a dense  $\Delta W$ :

$$\langle \Delta W_i, \Delta W_j \rangle_F = s_i s_j \text{tr}((B_i^\top B_j)(A_j A_i^\top)).$$

This turns an otherwise intractable sweep — eight experts  $\times$  nine epochs  $\times$  196 modules — into seconds of CPU, and we verified it against the dense computation to  $5.6 \times 10^{-16}$ . The identity holds only for vanilla LoRA (no DoRA, no rsLoRA), so the implementation refuses ragged or mixed-rank banks rather than trusting it.

We report three quantities. *Sign conflict* is the fraction of weight coordinates where two experts’ task vectors disagree in sign; TIES deletes exactly those, so a higher rate means more of the update is discarded at merge time. *Cosine* is the mean pairwise angle between task vectors.  $\|\Delta W\|$  is the mean Frobenius norm per module — how far each expert moved.

### 6.2 The interference mechanism is real

Prior work predicts that training experts to their individual optima raises interference and degrades merging [1]. On our eight-expert bank trained to three epochs, the mechanism is *absent*: sign conflict falls from 0.4016 to 0.3905 and TIES retention rises from 0.854 to 0.861. Our experts are under-trained relative to where interference appears. Extended to nine epochs, it reverses exactly as predicted — the task vector doubles in norm (0.291  $\rightarrow$  0.617), the experts rotate apart (cosine 0.580  $\rightarrow$  0.514), and sign conflict climbs from 0.4011 to 0.4254, plateauing around epoch 6 (Table 3).

### 6.3 The accuracy consequence does not follow

Mechanism is not consequence. On the same nine-epoch bank, with expert count and epoch held uniform across all eight experts, **training the experts longer made the merge better**: pooled over the six conditions, epoch 9 beats epoch 1 by +3.1 points [+0.0, +6.1] for DARE-TIES and +1.6 [-0.4, +3.5] for TIES, improving on eleven of twelve method  $\times$  condition pairs. In absolute terms the DARE-TIES merge moves from -2.6 points below the clean-code control at epoch 1 to +0.5 at epoch 9.



**Table 3: Task-vector geometry, eight experts, Python. Both banks contain the same eight conditions; they are separate training runs under different schedules, so a given epoch index is not the same point in both.**

| first epoch → last             | 3-epoch bank                   | 9-epoch bank                   |
|--------------------------------|--------------------------------|--------------------------------|
| $\ \Delta W\ $                 | 0.299 → 0.371                  | 0.291 → 0.617                  |
| $\cos(\Delta W_i, \Delta W_j)$ | 0.584 → 0.592                  | 0.580 → 0.514                  |
| <b>sign conflict</b>           | 0.402 → 0.391 ( <i>falls</i> ) | 0.401 → 0.425 ( <i>rises</i> ) |
| TIES keep rate                 | 0.854 → 0.861                  | 0.853 → 0.834                  |
| verdict                        | no interference growth         | interference grows             |

So the geometry is real — the task vectors do diverge and sign conflict does rise — but at LoRA rank 32 that interference does not cost merged accuracy, and the direction of the effect is opposite to the prediction. If anything the extra training buys more than the interference costs. Two things bound this: the effects are marginal at 40 programs, and an unexplained residual remains — the sign election retains 86% of task-vector magnitude, yet the resulting merge\_ties patch has a norm only 0.19× a single expert’s, which sign conflict alone does not account for.

## 7 MIXING EXPERTS WHERE NO SINGLE EXPERT IS RIGHT

### 7.1 The regime the router cannot reach

A saturated router (§4) means the choice of expert is solved on single-transformation inputs. The remaining headroom is therefore entirely on inputs where *no* single expert is correct. Stacked (composite) conditions manufacture exactly that: a variant that has been renamed *and* flattened contains two mechanisms, so a hard router must be wrong about one of them and a soft mixture can be right about both.

We mix in **activation space** rather than weight space:

$$h = Wx + \sum_e a_e(x) (\alpha/r) B_e A_e x.$$

This is exact, requires no per-item merge rebuild, and does not grow the effective rank of any single application. Only a 2.77M-parameter gate trains, against a frozen 295M expert bank (a ratio of 0.0094) and a frozen 1.5B base.

### 7.2 The ladder, and why each rung exists

mole\_uniform pins the gate at 1/8: it is the primary fixed-mixture contrast because it differs from the learned gate in *exactly one* way, the weights, running identical code with identical numerics. (The best weight-space merge differs in three ways at once — fixed versus learned, weight versus activation space, pruned-and-elected versus exact — which is why it is a reference rather than a comparator.) mole\_random freezes the gate at random initialisation and is the control that decides what the headline may claim: if the learned gate does not beat it, the gain came from having eight experts resident at effective rank 256, not from routing.

### 7.3 Result: the right sign everywhere, significance nowhere

Against the base model through the same engine, every mixture arm gains substantially (+8 to +28 points). Against mole\_uniform — the contrast that isolates routing — the learned gate is positive on **eight of eight** conditions: +7.4 on C\_L2\_S4, +4.0 on C\_L1r\_S1, +3.4, +2.8, +2.7, +1.7, +0.7, +0.6 elsewhere. Pooled, that is +3.3 points [-0.4, +7.5] over the six composites and +2.9 [-0.0, +6.2] ( $p = 0.059$ ) over all eight conditions. **No individual cell survives FDR correction**, and the pooled interval touches zero.

Eight-of-eight positive is itself unlikely under a null of no effect, and the per-condition effect sizes are of a plausible magnitude. But 40 programs cannot resolve a 3-point effect, and we decline to claim one on a sign pattern. This is the single largest reason the paper’s thesis slot is empty.

### 7.4 The control that would settle it is inert

While preparing this draft we found that mole\_random and mole\_uniform produce *bit-identical* output on all 1,299 evaluated items — identical generated strings, identical correctness vectors, identical format-failure rates. This is not near-uniform softmax at initialisation; it is the same gate.

The cause is an ordering dependency. The ladder runs its arms in sequence, and mole\_uniform runs first, replacing the learned gate module with a constant-gate module whose routing vector is a registered *buffer*. The mole\_random arm then re-seeds and re-initialises by iterating the gate’s *parameters* — of which a constant gate has none. The loop body never executes and the uniform gate survives into the arm meant to randomise it. We confirmed this directly: the constant gate exposes zero parameters, and running the re-initialisation loop against it leaves the routing vector at 1/8 for all eight experts. The neighbouring learned-gate branch guards against precisely this ordering hazard, with a comment saying arms must not depend on the order they run in; the control branch was not given the same guard.

Consequently the comparison reported above is mole\_router versus mole\_uniform *twice*, and **the residency question — is the gain from routing or merely from having eight experts resident? — is unanswered**. By the pre-registered rule that this control decides what the headline may say, no mixture claim is available. Repairing it requires no retraining and one evaluation pass (Table 4, item 5).

## 8 CONFOUNDS IN OUR OWN DESIGN

We report these because each materially qualifies a number above.

*Unequal expert training in the original merges.* Checkpoint selection chose *different* epochs per condition (epoch 1 for two experts, 2 for two, 3 for four), so every merge in Table 2 combines task vectors of unequal training. The uniform-epoch sweep in §6 removes this confound, and its within-sweep epoch deltas — not the comparison to Table 2 — are the controlled contrast.

*The quarantine budget was spent incrementally.* The protocol grants H1 exactly two evaluation passes, one pilot and one final. The access log records 3 pilot and 88 final read events spread over four sittings while arms were still being chosen. No H1 data entered

training and all four enforcement layers held — but “one frozen final pass” is not what happened, and the epistemic guarantee is weaker than the protocol claims. Any further H1 evaluation is a third pass and we will declare it as such. This is why Table 2 has — in the H1 column for the arms built after the budget was exhausted, rather than a number.

*Two of our own reports disagreed with the files.* While assembling this draft we found an internal report quoting a router training epoch from the wrong language and a composite acceptance count off by two programs. Both were resolved in favour of the result files. Every figure in this paper is recomputed from per-cell trial records; none is transcribed from a prior report.

## 9 FUTURE EXPERIMENTS

The programme below is what the empty thesis slot is waiting on. Each item is built and configured; none is speculative. Gates are stated in advance so that a negative outcome is reportable rather than merely disappointing.

*One hypothesis worth stating in advance.* Router saturation and merge failure may be two symptoms of one cause. Over-specialised experts are trivially distinguishable — hence easy to route — and mutually interfering — hence hard to merge. The joint prediction is that deliberately *under*-trained experts should be harder to route and easier to merge, tracing out a frontier between the two strategies. Neither the routing literature nor the merging literature makes this prediction, because each studies one half. We have both systems built on one bank, which is what makes item 8 cheap. **[CLAIM SLOT: this is the most promising route to a thesis; it is a hypothesis, not a result]**

*Two design defects we caught before spending GPU time,* recorded because they are the failure mode this kind of study is most exposed to. The mixture gate was initially wired to train on a *different task* from the one it is scored on — code rewriting rather than output prediction — which would have converged cleanly and produced a plausible number answering no question. And merge-search candidate names did not encode the other experts’ checkpoints, so two distinct merges shared a name and a mid-search restart would have silently scored the wrong one. Both defects share a shape: an identifier that does not encode what actually varies. Neither the test suite nor the manifest checker could catch them, because both check what is declared rather than what is meant.

## 10 THREATS TO VALIDITY

*Construct.* Output prediction is one operationalisation of comprehension, chosen because it is executable and therefore not gradeable by opinion. It does not measure whether a model could *explain* the code.

*Internal.* §8 lists the confounds we know of; §7.4 is an inert control we found only by comparing raw outputs, and its existence is evidence that others may remain. Splits partition by program, never by row, so an obfuscated variant of a training program cannot appear in the test set.

*Statistical.* The combination arms live on 40 Python and 30 JavaScript programs. Prior seed-stability analysis on this corpus puts run-to-run noise near 0.6 points in Python and 2.0 in JavaScript at much larger  $n$ ; at this  $n$  the bar is higher. The ordering of the arms is solid; individual cell values are indicative. Every combination arm is single-seed (Table 4, item 1).

*External.* One base model at one scale (1.5B), one task, two languages, one held-out obfuscator. Mixture and merging behaviour may be scale-dependent, and the oracle-prompt comparison was designed with larger models in mind (item 7).

## 11 CONCLUSION

Modularity is the natural engineering answer to a model that needs one specialist per code transformation, and we measured what it is worth. A router that identifies the transformation perfectly buys only the specialists’ own gains; weight-space merging gives most of them back; selecting expert checkpoints for mergeability rather than for accuracy changes nothing; and one previously catastrophic arm was a scaling defect that a format-failure diagnostic caught and accuracy alone would not have. The mechanism prior work blames for merge failure reproduces in our task vectors and does not produce the accuracy cost it predicts.

What remains open is the one regime where the question is not already answered: inputs where no single expert is correct. There, a learned gate is ahead of a fixed uniform mixture on every condition we measured and on none of them significantly — and the control that would tell us whether that is routing or merely capacity does not currently work. **[CLAIM SLOT: headline]** Section 9 says exactly what would settle it.

## REFERENCES

- [1] Stefan Horoi, Guy Wolf, Eugene Belilovsky, and Gintare Karolina Dziugaite. 2025. [VERIFY TITLE – arXiv:2506.14126]. *arXiv preprint arXiv:2506.14126* (2025). arXiv:2506.14126
- [2] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations (ICLR)*. arXiv:2106.09685 [cs.CL]
- [3] Chengsong Huang, Qian Liu, Bill Yuchen Lin, Tianyu Pang, Chao Du, and Min Lin. 2024. LoraHub: Efficient Cross-Task Generalization via Dynamic LoRA Composition. In *Conference on Language Modeling (COLM)*. arXiv:2307.13269
- [4] Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Ludwig Schmidt, Hananeh Hajishirzi, and Ali Farhadi. 2023. Editing Models with Task Arithmetic. In *International Conference on Learning Representations (ICLR)*. arXiv:2212.04089
- [5] Jack Le, Anh H. N. Nguyen, and Tien N. Nguyen. 2026. Do Machines Struggle Where Humans Do? LLM and Human Comprehension of Obfuscated Code. *arXiv preprint arXiv:2606.31725* (6 2026). arXiv:2606.31725 [cs.SE]
- [6] Dengchun Li, Yingzi Ma, Naizheng Wang, Zhengmao Ye, Zhiyuan Cheng, Yinghao Tang, Yan Zhang, Lei Duan, Jie Zuo, Cal Yang, and Mingjie Tang. 2024. MixLoRA: Enhancing Large Language Models Fine-Tuning with LoRA-based Mixture of Experts. *arXiv preprint arXiv:2404.15159* (2024). arXiv:2404.15159
- [7] Anh H. N. Nguyen, Jack Le, Ilse Lahnstein Coronado, and Tien N. Nguyen. 2026. The Effect of Code Obfuscation on Human Program Comprehension. *arXiv preprint arXiv:2603.07668* (3 2026). arXiv:2603.07668 [cs.SE]
- [8] Serge Lionel Nikiema, Jordan Samhi, Micheline Bénédicte Moumoula, Al-bérick Euraste Djiré, Abdoul Kader Kaboré, Jacques Klein, and Tegawendé F. Bissyandé. 2025. Using Contrastive Learning to Improve Two-Way Reasoning in Large Language Models: The Obfuscation Task as a Case Study. *arXiv preprint arXiv:2509.05553* (2025). arXiv:2509.05553 [cs.SE]
- [9] Jonas Pfeiffer, Aishwarya Kamath, Andreas Rücklé, Kyunghyun Cho, and Iryna Gurevych. 2021. AdapterFusion: Non-Destructive Task Composition for Transfer Learning. In *Proceedings of the Conference of the European Chapter of the Association for Computational Linguistics (EACL)*. arXiv:2005.00247
- [10] Baptiste Roziere, Marie-Anne Lachaux, Marc Szafraniec, and Guillaume Lample. 2021. DOBF: A Deobfuscation Pre-Training Objective for Programming

**Table 4: The specified programme. “Decides” names the claim each run adjudicates.**

| # | run                                                      | isolates                                                          | decides                                                                     |
|---|----------------------------------------------------------|-------------------------------------------------------------------|-----------------------------------------------------------------------------|
| 1 | Second seed for every combination arm                    | run-to-run noise, currently unmeasured for these arms             | whether any $\pm 3$ -point effect in Table 2 is real                        |
| 2 | Composite grid at corpus scale ( $\approx 550$ programs) | statistical power; 40 programs cannot resolve 3 points            | the §7 effect, which is the thesis                                          |
| 3 | mole_random with the ordering hazard repaired            | routing versus rank-256 residency                                 | <b>what the headline may say (§7.4)</b>                                     |
| 4 | Merge density sweep {0.3, 0.5, 0.7}                      | whether “merging underperforms” is a claim about density 0.5 only | the scope of §5                                                             |
| 5 | Hard-router rung of the mixture ladder on composites     | mixing versus dispatch, holding the engine fixed                  | whether composites are <i>compositional</i> or merely harder                |
| 6 | Router entropy on the held-out obfuscator                | how a saturated router fails out of distribution                  | a deployment-relevant claim; <b>counts as a third H1 pass</b>               |
| 7 | Replication at 7B                                        | scale dependence of every claim here                              | whether oracle prompting becomes competitive, i.e. “knows how but not when” |
| 8 | Undertrained-expert bank, routed and merged              | the joint prediction below                                        | a mechanism spanning both halves of the paper                               |

Languages. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2102.07492 [cs.CL]

- [11] Hanzhuo Tan, Qi Luo, Jing Li, and Yuqun Zhang. 2024. LLM4Decompile: Decompile Binary Code with Large Language Models. In *Findings of the Association for Computational Linguistics (EMNLP Findings)*. arXiv:2403.05286 [cs.PL]

- [12] Mitchell Wortsman, Gabriel Ilharco, Samir Yitzhak Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S. Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, and Ludwig Schmidt. 2022. Model Soups: Averaging Weights of Multiple Fine-Tuned Models Improves Accuracy Without Increasing Inference Time. In *International Conference on Machine Learning (ICML)*. arXiv:2203.05482

- [13] Xun Wu, Shaohan Huang, and Furu Wei. 2024. Mixture of LoRA Experts. In *International Conference on Learning Representations (ICLR)*. arXiv:2404.13628

- [14] Prateek Yadav, Derek Tam, Leshem Choshen, Colin Raffel, and Mohit Bansal. 2023. TIES-Merging: Resolving Interference When Merging Models. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2306.01708 [cs.LG]

- [15] Le Yu, Bowen Yu, Haiyang Yu, Fei Huang, and Yongbin Li. 2024. Language Models are Super Mario: Absorbing Abilities from Homologous Models as a Free Lunch. In *International Conference on Machine Learning (ICML)*. arXiv:2311.03099 [cs.CL]